

CSE465 Mobile Computing

Spring, 2026, Programming Assignment #2

Due: April 26, 2026 (Sun), 23:59 KST

Instructor: Taesik Gong (taesik.gong@unist.ac.kr), TA: Ahin Lee (ahin@unist.ac.kr)

Submission Instructions. You shall submit this assignment as a single ZIP file named PA2_StudentNumber_Name.zip (e.g., PA2_20261234_GildongHong.zip) on Blackboard. The ZIP file must include (1) the mobile app deliverable, (2) a written report, and (3) a demo video.

For Android, submit an APK file named PA2_App_StudentNumber_Name.apk. For iOS, submit the full Xcode project folder compressed as PA2_App_StudentNumber_Name.zip; do not submit only the .xcodeproj file.

The report must be submitted as PA2_StudentNumber_Name.pdf, and the demo video as PA2_Demo_StudentNumber_Name.mp4. All figures and text in the report must be clearly readable, and the video must open and play properly. If any file cannot be opened, is unreadable, or does not follow the required format, no points will be awarded.

Collaboration Policy. You are welcome to discuss the assignment with your classmates to better understand the concepts. However, all code, explanations, figures, screenshots, videos, and written materials that you submit must be entirely your own work. Copying another student's implementation or report, or directly sharing your own work with others, will be considered plagiarism and may result in penalties for all students involved.

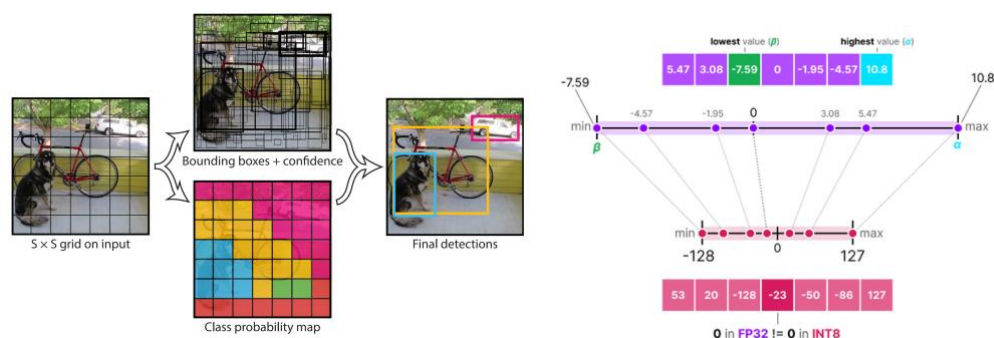
0. Background

Object detection enables a system to localize and classify objects in images or video streams and is widely used in mobile applications such as assistive computing, smart cameras, and context-aware services. However, deploying such models on smartphones is challenging due to limited compute resources, memory, and battery capacity. Therefore, mobile systems must balance accuracy, latency, and throughput to achieve practical performance.

Recent work in mobile vision highlights the importance of combining real-time detection with user-driven interaction. In particular, integrating voice-based input with on-device inference improves usability and enables efficient target object search in real-world scenarios [1]. This motivates the design of interactive, rather than purely passive, mobile vision systems.

Another key challenge is model efficiency. Techniques such as quantization and precision scaling (FP32, FP16, INT8) reduce computational cost and memory usage, but introduce trade-offs in accuracy. Understanding these trade-offs is critical for efficient on-device deployment.

In this assignment, you will build a voice-driven object detection system using a YOLO-based model [3] (yolo26n). You will deploy the model in multiple precision formats and evaluate their impact on latency, FPS, model size, and accuracy, while implementing the full mobile pipeline including camera input, voice commands, on-device inference, visualization, and performance analysis.



References

[1] Rahman, F.A.; Pusparani, F.A.; Yeoh, W.L.; Fukuda, O. Smartphone-Based Seamless Scene and Object Recognition for Visually Impaired Persons. *Information* 2025, 16, 808. <https://doi.org/10.3390/info16090808>

[2] Grootendorst, M. (2024, July 22). A visual guide to quantization: Demystifying the compression of large language models. Exploring Language Models. <https://newsletter.maartengrootendorst.com/p/a-visual-guide-to-quantization>

[3] Redmon, Joseph, et al. "You only look once: Unified, real-time object detection." CoRR, abs/1506.02640 (2015).

1. Design and Implementation of a Voice-driven Lightweight Object Detection System using YOLO.

In this assignment, you will design and implement a smartphone-based object detection and benchmarking system. You are expected to complete the full pipeline, including test-scene target-object definition, model integration, voice-based target selection, runtime precision switching, performance logging, and quantitative evaluation. At a minimum, the system should correctly demonstrate target-aware object detection in the demo video and provide a quantitative summary in the report (e.g., target detection success rate, per-class detection rate, latency table, FPS comparison, or a simple summary of detection results).

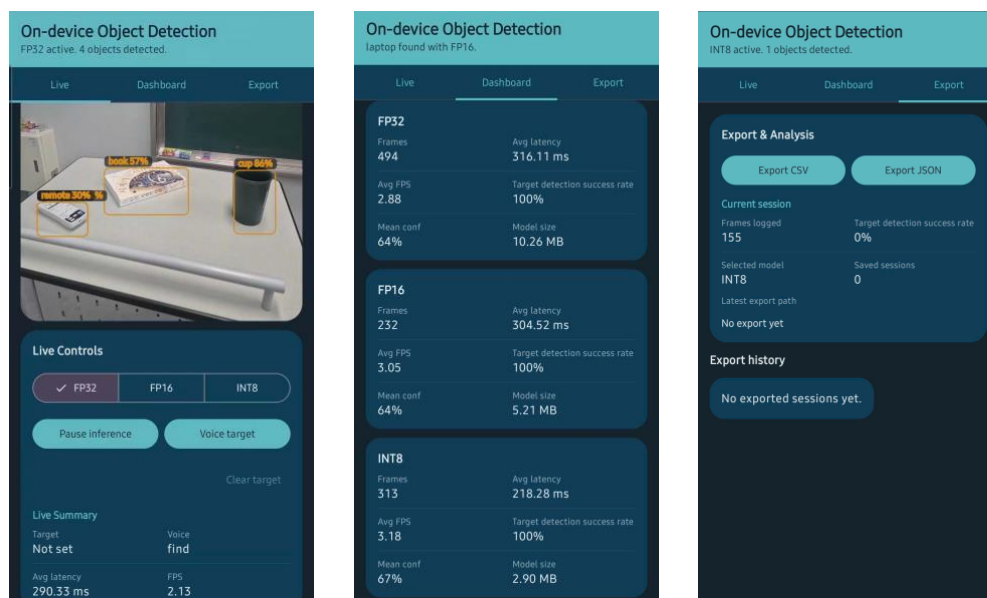
Before describing the detailed steps of the assignment, note that your final application must include three main functions: (1) a live detection interface, (2) a benchmark dashboard, and (3) an export and analysis interface. The details of these functions are described below. Your final application must include the following three main screens:

1) Live Detection Screen. This screen is used to run the object detector in real time. It should allow the user to view a camera preview, overlay detected bounding boxes and class labels, start or stop inference, select the model precision, trigger voice input, display latency and FPS and display the currently selected target object on the screen.

2) Benchmark Dashboard Screen. This screen is used to compare runtime behavior across FP32, FP16, and INT8 models. It should display at least the average inference latency, FPS, and model size. You are encouraged to additionally show battery current raw value, p95 latency, memory usage, device thermal state, input resolution, and delegate type.

3) Export and Analysis Screen. This screen is used to save logs for later quantitative analysis. It should allow the user to export CSV or JSON files containing timestamped inference results and system measurements.

Example of App Screens.



1. Voice Command Parsing and Target Object Selection.

Implement voice-based interaction to allow users to specify the target object while the camera is running. Spoken commands (e.g., "find bottle," "target person," "detect chair") should be parsed and mapped to object classes supported by the model.

When a target is set, the user interface must clearly indicate the current target and visually distinguish target detections from non-target detections. You may additionally provide feedback (text or audio) such as target found, target not found, or unsupported class.

In your report, describe:

- command formats and parsing strategy
- mapping logic between text and model classes

2. YOLO26n TensorFlow Lite Model Quantization.

You will use the pretrained YOLO26n model as the baseline detector (see: YOLO26: <https://docs.ultralytics.com/models/yolo26/>) and apply different precision formats to support efficient on-device inference. Specifically, you will work with FP32 (baseline), FP16, and INT8 versions of the model. This part of the assignment focuses on understanding how model quantization and precision scaling affect system performance under mobile constraints.

In your report, you should describe the quantization process used to obtain the FP16 and INT8 models using a post-training quantization (PTQ) approach. Clearly explain how the original FP32 model is converted into lower-precision formats, including any tools, conversion steps, and calibration procedures if applicable.

You should then analyze how each precision format behaves in a mobile environment, including its impact on inference latency, FPS, model size, and energy consumption. Based on your experiments, discuss the trade-offs between efficiency and accuracy across different precision levels.

3. Model Integration and Precision Switching.

You will integrate the YOLO26n TensorFlow Lite models into an on-device object detection pipeline. The system should support runtime switching between FP32, FP16, and INT8 models without requiring a rebuild. The pipeline should capture frames from the device camera, preprocess them to the model input format, run TensorFlow Lite inference, and display detection results in real time.

4. Performance Dashboard, Exported Logs, and Accuracy Analysis.

Measure and display at least the following values during runtime: average inference latency, current or rolling FPS, model precision, model file size, battery current raw value, and the currently selected target class. You are encouraged to additionally track p95 latency, memory usage, delegate type, and thermal status if available on the device.

Your application must export logs in CSV or JSON format. At minimum, the logs should include timestamp, frame index, selected model precision, latency, FPS, target class, predicted classes, confidence scores, and system measurements. The exported values should be sufficient to perform later analysis of model-wise accuracy and efficiency.

5. Demonstration of the Final App.

Your final application should successfully demonstrate live on-device detection and voice-based target selection under real scene conditions. In the report and demo video, you should clearly show the app running, the voice command input, bounding-box overlays, model switching between FP32, FP16, and INT8, dashboard updates, and the exported results or a short analysis summary.

2. Report and Demo Video

In this assignment, your implementation must be accompanied by both a written report and a demo video.

1. Report. (Maximum 5 pages, single-column, with font size no smaller than 10 pt.)

Your report must include a clear overview of the system, the design of the voice command interface, screenshots of the application, a comparison of the YOLO26n FP32, FP16, and INT8 models, performance dashboard measurements, and an analysis of the exported logs. The report should not be written as a simple checklist of components. Instead, it should explain the rationale behind your design choices and present your implementation and analysis in a clear and logical manner.

For your convenience, we recommend organizing your report using the following structure:

- 1. Introduction or Assignment Overview**
Briefly describe the goal of the assignment and provide an overview of your system.
- 2. Voice Interaction and Target Selection**

Explain how voice commands are processed, how text is mapped to object classes, and how the user interface reflects the selected target.

3. Model Quantization

Describe the quantization process used to obtain FP16 and INT8 models from the FP32 baseline, including tools, conversion steps, and any calibration procedures.

4. System Design and Implementation

Explain the overall detection pipeline, including camera input, preprocessing, inference, post-processing, visualization, and runtime model switching.

5. Performance Evaluation and Results

Present your experimental setup and results, including latency, FPS, model size, and other measured metrics. Include screenshots or logs if appropriate, and discuss observed differences across precision formats. Accuracy analysis may be approximate.

6. Discussion and Conclusion

Summarize the key findings from your experiments, especially the comparison between FP32, FP16, and INT8 models. Discuss which configuration is most suitable for mobile deployment and why. You may also describe limitations of your system and possible improvements.

2. Demo Video.

The demo video must clearly show that your application is functioning as intended. At a minimum, the video should include the app being launched and executed, real-time camera-based object detection, and voice-based target object selection. The video should demonstrate that the system correctly responds to spoken commands and highlights the selected target object in the detection results.

The video must be at least 60 seconds long. The demonstration may be recorded using screen capture. The video should clearly show the app interface, including the camera preview, precision switching, detection results, and user interactions. The demonstration should make it easy to verify that the system is functioning correctly, including voice command input, target selection, and real-time detection results.

The demo video must also include a test scenario with at least three distinct objects in the scene, including the selected target object. The demo should also include runtime switching between FP32, FP16, and INT8 models, along with visible changes in the performance dashboard (e.g., latency, FPS, model precision). If applicable, briefly show exported results or a summary of performance comparison.

The video must clearly demonstrate the actual operation of your system. If the video does not appear to reflect a working implementation, or if it is suspected to be misleading or fabricated, no points will be awarded for this assignment.

3. Interesting Voice Command Extension (up to 20 extra points)

You may receive extra credit by adding an additional interesting feature based on voice commands beyond the required functionalities. To receive extra credit, you should clearly define the new feature and demonstrate how it extends the existing system.

Some examples of suitable extensions might include custom voice commands for controlling system behavior (e.g., adding support for new target objects through custom voice commands, filtering specific object categories, or triggering alerts), or interactive features that enhance usability through voice input. These are only examples to help you think about the problem, and we encourage you to come up with your own creative idea.

Please note that simply adding another command is not sufficient. You should explain how your feature differs from the required functionalities, describe how the voice input is processed and integrated into the system, and demonstrate how it affects system behavior. Your extension should include meaningful system-level reasoning, and the additional feature should be implemented, demonstrated, and analyzed together with the required system.

4. Grading (Total 100 points + up to 20 extra points)

(1) Real-time detection and voice-based target selection: 20 points

(2) Model quantization and performance analysis: 50 points

(3) App completeness, report quality, and demo video: 30 points

(4) Interesting voice command extension: up to 20 extra points